Reordering Data Movement and Element Size-Reducing Operations in Triton IR for Efficient GPU Kernels

Pushpendra Mehra

June 19, 2026

Final MTech Project Report

Abstract

Programs running on GPUs frequently combine data-movement operations (reshapes, broadcasts, transpositions, concatenations) with data size–reducing operations (precision truncation, quantization, integer narrowing). When the size-reducing operation is applied *after* the data-movement operation, the compiler propagates the wide pre–size-reduction values through the mover, increasing register pressure, spill traffic, and DRAM bandwidth consumption in memory-bound kernels.

We designed and implemented `DowncastReorderOptimizer`, a dependency-aware optimization at the Triton IR level that automatically identifies (mover, element size-reducer) pairs and rewrites the IR to hoist the size-reducing operation upstream of the data-movement operation whenever it is semantically safe. The pass uses worklist-driven fixed-point iteration, an (OperationName, ResultType) de-duplication(dedup) cache for multi-consumer fan-outs, and a small set of eligibility guards that keep the transformation value-preserving on every DAG topology. The pass handles six mover ops and six element size-reducer kinds in the Triton IR.

We evaluate the pass on a microbenchmark suite covering linear mover chains, two- and three-way diamond fan-outs, nested cats, multi-level diamonds, and Y-shape upstreams. The pass is bit-exact between without our optimization and with our optimization on every output of every kernel (SHA-256-verified).

We report two distinct performance numbers per kernel. The "best-vs-best speedup" compares the fastest executing tile-size without our optimization to the fastest executing tile-size with our optimization and answers "does the pass make a tuned workload faster?". The "per-tile peak rescue" compares execution time without our optimization and with our optimization at the same tile-size and answers "how much does the pass save a developer or auto-tuner committed to a sub-optimal tile?". The two numbers measure different things and the difference is large: on linear mover chains the best-vs-best gain is 1.01–1.04× even when the per-tile peak rescue reaches 8.04×, because the per-SM register file is large enough that some small tile always remains fast in either pass state. The pass's impact on those chains is to roughly double the viable-tile set - the set of tile shapes that sit on the non-spilling side of the register-fit boundary. On multi-consumer diamond fan-outs, the dedup cache changes structural register usage and the best-vs-best gain becomes meaningfully non-trivial (1.06–1.24×, largest at the cache-split diamond). A small number of regressions appear on diamonds at tile shapes where every cloned narrow chain still spills; a register-pressure gate at the rewrite point is the natural mechanism to suppress these.

A single quantitative model $\Delta t_{\text{predicted}} = (\Delta\text{DRAM bytes})/(\text{ON DRAM rate})$ accounts for 66–100% of the wall-time delta on 9 wins and 58–70% on two regressions across 11 configurations spanning 6 kernels. The regression cases confirm the mechanism in reverse: when narrowing duplicates a chain that still does not fit in registers, the same model predicts the slowdown. The pass is therefore correct on every DAG, insurance against the spill cliff on every kernel, and a workload-level accelerator where structural duplication (diamonds) changes the live-range graph in a way that lifts the best tile too.

1 Introduction

Triton is a domain-specific compiler that lowers a Python-embedded tile-level programming model into device-specific GPU code [1]. Its abstractions over thread mapping, shared memory, and instruction scheduling have made it a popular tool for machine learning kernel authors. Many memory-bound GPU

kernels — including the precision-mixing chains common in modern ML workloads — can be bottlenecked by data movement rather than arithmetic, in which case minimizing the bytes carried through layout transformations becomes a useful lever [2].

A common computation pattern in kernels of this kind is the following: a *data-movement* operation (e.g., concatenation, transpose, reshape, broadcast etc.) is followed by a *data size–reducing* operation (e.g., `truncate` from fp32 to fp16, int8 quantization, narrowing `tt.fp_to_fp` for fp8 etc.) that discards a substantial fraction of the bit-width of every element. When the size-reducing operation is applied *after* the mover, the wide pre–size-reduction intermediate lives through the mover; this intermediate is exactly the live tile that the register allocator must keep resident, and exactly the working set that hits the spill regime when it exceeds the per-SM register file.

Motivating example: The TTIR fragment below shows a two-dimensional tile undergoing a transpose followed by an fp32 to fp16 truncation:

```
%0 = tt.trans %arg0
     : tensor<32x32xf32> -> tensor<32x32xf32>
%1 = arith.truncf %0
     : tensor<32x32xf32> -> tensor<32x32xf16>
```

The `tt.trans`(transpose) operation preserves both the number of elements and the element precision, even though the value will be truncated to fp16 immediately afterwards. The materialized intermediate occupies twice as many registers (in fp32) as necessary. A semantically equivalent reordering hoists the truncation upstream of the mover:

```
%0 = arith.truncf %arg0
     : tensor<32x32xf32> -> tensor<32x32xf16>
%1 = tt.trans %0
     : tensor<32x32xf16> -> tensor<32x32xf16>
```

Now the `tt.trans` operates on fp16 data: registers per live element halve, the intermediate is narrower throughout, and the kernel can avoid spill regimes that the wide form would force.

This work designs, implements, and evaluates a Triton compiler pass (`DowncastReorderOptimizer`) that automatically performs this reordering under a precise set of legality constraints. The pass is implemented at the Triton IR(ttir) level of the Triton compilation pipeline, using MLIR's IR rewriting infrastructure [3].

Contributions:

1. A precise taxonomy of Triton-IR *movers* and *element size-reducers*, together with a per-mover classification by effect on live element count (Section 3).
2. `DowncastReorderOptimizer`: a worklist-driven TTIR transform that sinks element size-reducing operations upstream of six data-movement operations, with an `(OperationName, ResultType)` dedup cache that handles multi-consumer diamond DAGs and an all-consumers-reducible(all consumer should be element size-reducing operations) guard that prevents unsafe duplication (Section 5).
3. A microbenchmark suite of 14 kernels covering linear chains, two- and three-way diamonds, nested cats, multi-level diamonds, and Y-shape upstreams. Every output of every kernel is verified bit-exact between without our optimization and with our optimization via SHA-256 (Section 6).
4. A causal verification of the speedup mechanism: a single quantitative model $\Delta t_{\text{pred}} = (\Delta\text{DRAM bytes})/(\text{ON DRAM rate})$ that accounts for 66–100% of the wall-time delta on 9 wins and 58–70% on two regressions across 11 configurations on 6 kernels (Section 7). Both directions are verified: positive DRAM saved ⇒ speedup, negative ⇒ slowdown.
5. A characterisation of the pass's operating regime: the rewrite is value-preserving on every DAG topology, and produces measurable speedups across the suite, with the largest gains on size-changing movers where the rewrite relocates the register-fit boundary(explained later) by the bit-width factor (Section 6).

2 Background and Related Work

Selection pushdown and equivalent rewrites: Pushing size-reducing operations upstream of data-movement operations is a generalization of *selection pushdown* in relational query optimization [4]: an operation that discards data should be applied as early as possible to limit the volume processed downstream. The same principle appears as *deforestation* in functional array languages [5, 6] and as "slice sinking," "transpose sinking," or "reshape sinking" in contemporary tensor compilers.

Triton's compilation pipeline: The Triton frontend lowers the Python tile-level kernel into Triton IR(ttir), an MLIR dialect that retains tile-typed tensor operations with explicit ranked shapes and el-

ement types. `ttir` is then transformed into `ttgir` (TritonGPU IR), which lowers tile-level ops to hardware-aware layouts (`BlockedEncoding`, `MmaEncoding`, `SharedEncoding`) and inserts inter-warp synchronization. The TritonGPU pipeline lowers to LLVM IR (`llir`), then to PTX, and finally to `cubin`. `DowncastReorderOptimizer` sits at the `ttir` level, where the IR retains the def-use relationships and explicit element types needed for legality analysis without yet committing to hardware-specific layout decisions.

XLA's `algebraic_simplifier`. The most extensive production-quality library of such rewrites is XLA's `algebraic_simplifier` (more than 25 distinct size-reducer-outside / mover-inside rewrites; key handlers `HandleSlice`, `HandleReduce`, `HandleDynamicSlice`, `HandleGather`, `HandleDot`, `HandleConvolution`). XLA's framework distinguishes operations that *permute* their operand's elements (`Reshape`, `Reverse`, `Transpose`, `Sort`) from those that pick a *subset* and uses helper predicates to drive rewrite eligibility checks. The 5-step structural pattern XLA uses (match, bail-out, index/dimension remapping, build new HLO, replace) is the same shape we adopt for `DowncastReorderOptimizer`.

Where this work differs from XLA: XLA's rewrites are *shape rewrites*: pushing `Reduce` through `Transpose` requires re-permuting the reduction-dimension attribute; pushing `DynamicSlice` through `Transpose` requires `InversePermutation` and `Permute` on indices and sizes. `DowncastReorderOptimizer` solves a strictly easier *type-only* problem: the cast is elementwise and shape-preserving, so attribute carry-through is trivial and only the result types of the mover and its operands need to be reconstructed. Because the pass operates at the `ttir` level, the hardware-aware encodings introduced later by the TritonGPU lowering are not yet present in the IR, so the rewrite is not entangled with layout decisions taken further down the pipeline. We still run `verify()` on every newly-built mover before committing it as a safety check.

Other tensor compiler frameworks: MLIR's Linalg/StableHLO [3] canonicalization patterns, TVM's Relay `SimplifyExpr`, PyTorch/TorchInductor pattern-matched decompositions, and ONNX Runtime's graph optimizer all implement subsets of the size-reducer-outside / mover-inside rewrite family described above for XLA. Equality-saturation systems TASO [7], PET [8], and Tensat enumerate rewrites systematically rather than handcrafting them; this work takes the handcrafted route to stay within Triton's existing TTIR-level transformation framework. Quantization-aware compilation literature [9] discusses when integer quantization should be applied in the dataflow, but does not address its relative ordering with respect to layout transformations inside a single kernel.

Multi-user policy: XLA's policy when a producer has multiple users is to refuse by default. DowncastReorderOptimizer takes the opposite posture: when all users are size-reducing, the rewrite fires aggressively, duplicating the producer once per distinct (OperationName, ResultType) consumer key. A small number of diamond configurations in Section 6 show this aggressive default can be counterproductive when every cloned chain still spills; a register-pressure gate at the rewrite point is the natural mechanism to suppress those cases.

3 Problem Statement

3.1 Operation Classification

We classify TTIR ops into two disjoint sets by effect on live data volume.

Data-movement operations rearrange, replicate, or reorganize tensor data without reducing element bit-width. The pass handles six: `tt.cat` (concatenation along an existing axis, $n_{\text{out}} = n_A + n_B$), `tt.join` (stack along a new innermost axis, same accounting as cat), `tt.trans` (axis permutation, $n_{\text{out}} = n_{\text{in}}$), `tt.reshape` (shape reinterpretation, $n_{\text{out}} = n_{\text{in}}$), `tt.expand_dims` (insert size-1 axis, $n_{\text{out}} = n_{\text{in}}$), and `tt.broadcast` (replicate size-1 axes, $n_{\text{out}} = n_{\text{in}} \cdot \prod_{j \in B} d'_j$; the only mover with $n_{\text{out}} > n_{\text{in}}$).

The three layout-only movers (`trans`, `reshape`, `expand_dims`) permute or relabel a tile without changing the number of live elements, so the register allocator sees an isomorphic problem in both pass states; Patterns 4 (main body) and A.1 (appendix) confirm these produce SASS-level no-ops at the `transpose+truncate` and `reshape+quantization` chains. The size-changing movers (`cat`, `join`, `broadcast`) create a spill-fit boundary at the materialization point of their larger output tile; this is the boundary the rewrite crosses.

Element size–reducing operations decrease the bit-width of each element. The pass handles six:

`arith.truncf` (float truncation), `arith.trunci` (integer truncation), `arith.fptosi` and `arith.fptoui` (float-to-int casts, which always narrow when the destination is an $\leq$ 32-bit integer), narrowing `tt.fp_to_fp` (e.g., fp32 to fp16, fp32 to fp8E5M2; the latter is the only TTIR cast for the fp8 family), and has a narrowing guard so that source and destination are concrete `IntegerType`s and the destination is strictly narrower.

3.2 Inefficient Ordering Pattern

A commonly observed pattern is the chain

$$v_1 = M(v_0), \quad v_2 = R(v_1),$$

where M is a data-movement operation and R is a data size–reducing operation that consumes M's output. M operates on the full-precision value v_0 even though R immediately discards a fraction of v_1's precision; the live intermediate v_1 carries more bytes through the mover's lowering than the program semantically requires.

3.3 Desired Reordering and Legality

We rewrite the chain to

$$v_1' = R(v_0), \quad v_2' = M(v_1'),$$

so that M operates on the narrower value v_1'. The reordering is legal only if: (i) *Dependency preservation*: every user of v_1 in the original program is itself size-reducing with a destination type compatible with R's destination, so no consumer of the unreduced value escapes. (ii) *Shape compatibility*: applying R before M produces tensor shapes and indexing semantics consistent with the mover (trivially satisfied because R is elementwise and shape-preserving). (iii) *Semantic equivalence*: the bits produced by the rewritten chain are bit-exactly equal to those produced by the original. (iv) *No observable side effects* on either operation; all movers we handle are tagged `Pure` or `NoMemoryEffect` in TTIR, and all reducers are pure arithmetic.

3.4 Problem Definition

Given a Triton IR module, identify all instances of the inefficient ordering pattern above, decide for each instance whether the reordering is legal under conditions (i)–(iv), and apply the reordering in place such that the rewritten module remains well-formed at TTIR and continues through the standard Triton lowering pipeline (TTGIR, LLIR, PTX, cubin) without intervention.

4 Register Budget and the Spill-Fit Boundary

The speedup mechanism the pass relies on is mediated by the per-SM register file. On the Turing GPU (CC 7.5, e.g. RTX 2080 Ti), each SM has 65,536 32-bit registers; per-thread allocation is capped at 255 registers (an architectural limit). With W active warps per SM, the register-file constraint is $W \cdot 32 \cdot R_{\text{thread}} \leq 65{,}536$, i.e. $W \leq 64/R_{\text{thread}}$. Beyond $R_{\text{thread}} = 32$ every register costs warp residency, and at the per-thread cap of 255 only $W = 1$ warp fits (3.1% theoretical occupancy).

The spill regime: When live values exceed the register allocation, the excess is *spilled* to per-thread local memory (DRAM-backed, L1- and L2-cached). Three NCU signals identify the spill regime: the SASS-level `smsp__sass_inst_executed_op_local_{ld,st}.sum` counters (nonzero $\Rightarrow$ spilling); `long_scoreboard` (warps blocked on long-latency memory dependencies); and `lg_throttle` (warps blocked because the local/global memory pipeline is saturated). The fingerprint of spill traffic specifically is `lg_throttle` — spill volume is usually 10–100$\times$ the kernel's global traffic on the same kernel, so a spilling kernel saturates the local-memory pipeline.

Spill-fit boundary: The *spill-fit boundary* for a given chain is the largest tile size at which the kernel compiles to zero spill traffic with at least one warp active. For an fp32 tile the boundary sits roughly at $n \approx$ 16,384–32,768 depending on what else is live at the same program point. Narrowing the live tile by a $k\times$ bit-width factor extends the boundary by roughly k — fp16 is 2$\times$, i8 is 4$\times$, fp8e5 is 4$\times$. Spilling is a *phase transition*, not a smooth gradient: a single tile-size step crossing the boundary jumps from zero spill to tens of millions of local-memory requests, swinging wall-clock time by 5–8$\times$ while the algorithmic work only doubles. The pass earns its speedups by relocating this cliff through the bit-width factor; it earns its regressions when chain duplication pushes the cliff back in the wrong direction.

Materialization equivalence: A spill load/store pair across a program point is structurally equivalent to splitting the kernel at that point with a local-memory round-trip in between. A fully-spilling fused kernel and an unfused, per-op-materializing eager-PyTorch chain therefore occupy the same regime —

which is why the torch-eager baselines we report alongside each experiment are, at heavy-spill tiles, no faster than without our optimization and sometimes slower.

5 DowncastReorderOptimizer: Algorithm and DAG Handling

The pass is a single-file TTIR transform implemented in MLIR. It runs at the `ttir` stage of the Triton pipeline, before TTGIR lowering; at this level the IR retains explicit tensor shapes, element types, and SSA def-use relationships needed for legality analysis.

5.1 The Atomic Rewrite

For one (mover, element size-reducer) pair the rewrite proceeds in four steps.

1. Read the consumer's destination element type τ_{dst}.
2. For each of the producer's operands o_i (each a ranked tensor), construct a new ranked tensor type T_i' that has the same shape and encoding as o_i's type but element type τ_{dst}. Clone the consumer's reducer op (via `createSizeReducingClone`, which dispatches on op kind via a `TypeSwitch`) with o_i as input and T_i' as result type. The cloned reducer is the *same* kind as the original, applied earlier in the chain. For `tt.fp_to_fp` the clone copies `getRoundingAttr()`, which is the only attribute carry-through that matters.
3. Build a new data-movement op of the same kind as the producer, with the cloned narrow operands and a result type whose element type is τ_{dst}. All attributes of the original mover are carried over with `state.addAttributes(...)`. Run the MLIR verifier on the new op; on failure, erase it and bail.
4. Return the new mover's result. The old chain $o \to M_{\tau_{\text{src}}} \to R_{\tau_{\text{dst}}}$ becomes $o \to R_{\tau_{\text{dst}}} \to M_{\tau_{\text{dst}}}$. Same algorithm, narrower intermediate.

5.2 Eligibility Guards

The function `eligibleReducers` checks six conditions before attempting any rewrite on a producer.

(1) Producer is in `isSafeDataMovementOp` (one of the six movers). (2) All producer operands are defined in the same block, above the producer (block-local SSA dominance, checked with `isBeforeInBlock`). (3) `TransOp` producers have rank ≥ 2 (transpose of a rank-1 tensor is a no-op). (4) Producer result is a ranked tensor.

(5) **All-consumers-reducible bailout:** If *any* user of the producer is not in `isSizeReducingOp`, return empty immediately. This is the most important guard. When a non-reducer user exists, the rewrite would leave the original producer alive (because the producer is erased only under `use_empty()`), so the cloned narrow chain would run *in addition to*, not *instead of*, the original wide chain — the duplication strictly adds work without removing the wide intermediate.

(6) **Per-operand bitwidth check:** Every producer operand's element bitwidth must be strictly greater than the consumer's destination bitwidth. If any operand is already at the destination width, the consumer is skipped. This rules out rewrites that buy nothing.

5.3 Multi-Consumer Fan-out: The Dedup Cache

When a single producer has k size-reducing consumers, `applyMultiConsumerOptimization` iterates over them with a `DenseMap` cache keyed on (OperationName, ResultType). Consumers that match *both* components of the key share one rewritten chain; any consumer that differs in either component (a different reducer kind or a different destination type) gets its own rewritten chain.

Generically, a multi-consumer diamond on a mover produces one narrowed chain per distinct key, and the original wide producer is erased once every consumer has been redirected (under `use_empty()`). The kind component of the key is essential for correctness: two reducers can share the same MLIR result type yet differ in numeric semantics (for example, signed vs unsigned float-to-int casts both yield an 8-bit integer type but differ in how out-of-range inputs are encoded), so sharing one chain in such a case would produce wrong bits for one of the outputs.

5.4 Chain Propagation: The Worklist

A chain like `Cat` $\to$ `Trans` $\to$ `Trunc` requires two rewrites: push `Trunc` above `Trans`, then push the *new* narrowed reducer above `Cat`. `runOnce` uses an `SmallSetVector` worklist (insert order preserved, $O(1)$ dedup) to propagate the rewrite upstream within a single sweep.

The driver: (a) walk the module once and seed the worklist with every producer whose `eligibleReducers` returns a non-empty list; (b) pop a producer and re-check eligibility at pop time (earlier rewrites may have changed its user set); (c) before rewriting, capture the producer's upstream-mover defining ops; (d) apply `applyMultiConsumerOptimization`; (e) requeue the captured upstream movers — they may now satisfy the all-reducer guard because the cloned reducer became their new user. Termination follows because each successful rewrite strictly decreases the number of (mover, element size-reducer) pairs in the IR. `runOnOperation` calls `runOnce` up to 16 times as a defensive fixed point; in practice one sweep suffices on every kernel we evaluate.

5.5 Where the Pass Does Not Fire

By construction: (i) any producer with a non-reducer user (guard 5); (ii) widening casts (`isSizeReducingOp` returns false); (iii) cross-block producer-consumer pairs (block-local SSA check); (iv) per-operand bitwidth mismatches where any producer operand is already at the destination width (that particular consumer is skipped, not the whole producer).

The pass's policy is therefore deliberately simple: it fires whenever narrowing is locally safe (all consumers reduce, every operand has room to narrow). This is sufficient on every single-consumer chain and on most multi-consumer diamonds; a register-pressure check at the rewrite point is the natural next addition for the few diamond configurations where every cloned narrow chain still exceeds the register file.

6 Experimental Evaluation

6.1 Setup

All experiments use an NVIDIA RTX 2080 Ti (Turing, 68 SMs, CC 7.5, 11 GB GDDR6, peak HBM bandwidth $\sim$ 616 GB/s; driver 535.104.12), Triton 3.6.0 built from source with `DowncastReorderOptimizer` added to the Triton transforms directory, CUDA 12.2, `nsys 2023.2.3`, `ncu 2023.2.2`, and PyTorch 2.9.1+cu128. The pass is toggled by commenting or uncommenting its registration in the Triton Python pass-pipeline setup, then wiping the Triton cache between toggles. Timings are mean execution time averages from `nsys cuda_gpu_kern_sum` over 100 iterations between `cudaProfilerStart` and `cudaProfilerStop` after 3 warmup launches. NCU collection uses `-set full` or `-set detailed` under `sudo` for perf-counter access. Block sweeps cover $(\mathrm{BLOCK}_M, \mathrm{BLOCK}_N) \in \{64, 128, 256\}^2$ at $M = N \in \{4096, 8192\}$.

Workload class: All kernels operate on a single two-dimensional matrix $A \in \mathbb{R}^{M \times N}$ (or a pair A, B) loaded from global memory, transformed through a mover chain ending in a size-reducing cast, and stored to global memory. The kernels are deliberately microbenchmark-style: each program instance processes one tile, and the grid is sized to cover the full matrix. This isolates the effect of the rewrite from downstream kernels and removes confounding effects from inter-kernel synchronization, launch overhead variance, and cache thrashing across kernels. The two-dimensional matrix shape is an evaluation choice for these microbenchmarks; the pass itself is not restricted to rank-2 tensors. The eligibility checks (block-local SSA, all-consumers-reducible, per-operand bitwidth) and the rewrite logic operate on ranked tensor types of arbitrary rank, and the `tt.trans` guard explicitly admits any rank ≥ 2. Higher-rank tiles would extend the same mechanism, with the only architecture-dependent change being the register-fit boundary shifting with the per-element working set.

Measurement methodology: Mean execution time latency is the mean over 100 launches between `cudaProfilerStart` and `cudaProfilerStop`, after 3 warmup launches that are explicitly excluded by the profiler. Each mean execution time measurement is extracted from `nsys cuda_gpu_kern_sum` for the specific kernel name; this excludes host-side launch overhead and only counts on-device time. NCU launches profile a *single* invocation of each kernel under `ncu -set full` (or `-set detailed` for cross-config sweeps), with hardware counter values read once per launch and averaged across 4 launches by NCU's internal sampling.

6.2 Correctness

Bit-exact agreement between without our optimization and with our optimization (and against PyTorch references) was verified on every kernel by hashing every output tensor with SHA-256. For multi-output kernels (diamond fan-outs), each output is hashed independently. The pass is value-preserving across 14 kernels and the full block sweep at both matrix sizes; maximum $|\mathrm{dst} - \mathrm{ref}| = 0.0$ in every cell. This includes the `fp8e5` output of the three-distinct-kind diamond, the only kernel in the suite that exercises the `tt.fp_-`

`to_fp` clone branch and its `getRoundingAttr` carry-through.

6.3 Pattern 1: Concatenate – Truncate

Kernel: Two fp32 tiles a, b of shape $(\text{BLOCK}_M, \text{BLOCK}_N)$ are flattened, concatenated with `tl.cat` to a length-$2\,\text{BLOCK}_M\text{BLOCK}_N$ fp32 vector, truncated to fp16, reshaped to $(\text{BLOCK}_M, 2\,\text{BLOCK}_N)$, and stored. The TTIR fragment the pass acts on is:

```
OFF: %c = tt.cat %a, %b
          : tensor<NxNxf32> -> tensor<2NxNxf32>
     %d = arith.truncf %c
          : tensor<2NxNxf32> -> tensor<2NxNxf16>
ON:  %a' = arith.truncf %a
          : tensor<NxNxf32> -> tensor<NxNxf16>
     %b' = arith.truncf %b
          : tensor<NxNxf32> -> tensor<NxNxf16>
     %c  = tt.cat %a', %b'
          : tensor<NxNxf16> -> tensor<2NxNxf16>
```

The original fp32 `tt.cat` and `arith.truncf` are both dead after rewrite and are erased.

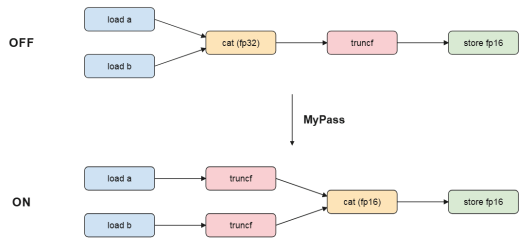

Figure 1: Operation flow for the Concatenate – Truncate kernel. Our optimization hoists the `truncf` cast upstream so that the `tt.cat` operates on fp16 inputs. The dedup cache emits one cloned `truncf` per cat operand; the post-cat live tile is now 2× narrower per element.

Headline numbers: *Best-vs-best speedup:* **1.04**× (fastest without our optimization tile $(64, 64)$ at $1523\,\mu$s vs. fastest with our optimization tile $(64, 256)$ at $1464\,\mu$s). *Viable-tile expansion:* the number of tile shapes within 20% of the best without our optimization latency rises from **3/9** to **6/9** — the pass roughly doubles the set of tile shapes that remain register-resident. *Per-tile peak rescue:* 3.90× at $(128, 256)$, where the wider intermediate exceeds the register file under without our optimization and the narrowed form stays within it under with our optimization. This figure measures the gain available to a developer or autotuner committed to a fixed tile shape, complementing the best-vs-best number. Across the 9-tile sweep: 12 tiles improve by $\geq 1.10\times$ at fixed tile shape, 6 are neutral, no regressions.

Mechanism at the peak-rescue tile $(128, 256)$**:** without our optimization sits at 64 regs/thread with 14.5 M spill loads and 94.1% occupancy; with our optimization crosses to 255 regs/thread, spill collapses 7.4×, instructions halve, and wall-clock matches the 3.90× nsys figure. Tile $(64, 64)$ fits in registers either way and is invisible at SASS. Full NCU and warp-stall tables in Appendix F. Torch-eager baseline `torch.cat(...).to(fp16)` is $3513\,\mu$s — best Triton with our optimization ($1464\,\mu$s) is 2.40× faster.

6.4 Pattern 2: Deep Mover Chain

Kernel: `load a, b` → `reshape` → `reshape` → `cat` → `reshape` → `trans` → `fptosi (i8)` → `store`. The truncation is sunk through four data-movement operations before landing at the `cat` operands.

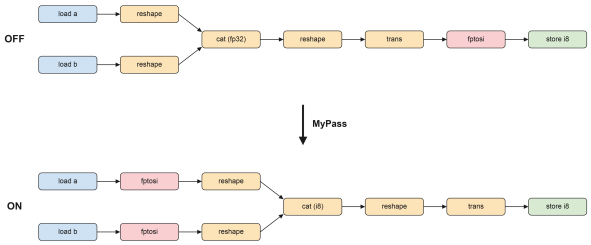

Figure 2: Operation flow for the Deep Mover Chain kernel. The worklist propagates the `fptosi` cast upstream through four data-movement operations in one sweep, landing on the fp32 loads. Every intermediate from the load onward is i8 (4× narrower per element).

Headline numbers: *Best-vs-best speedup:* **1.01**× (fastest without our optimization tile $(128, 64)$ at $1226\,\mu$s vs. fastest with our optimization tile $(256, 128)$ at $1214\,\mu$s). The fastest configuration is essentially unchanged. *Viable-tile expansion:* tiles within 20% of the best without our optimization latency rise from **5/9** to **8/9** — the pass moves nearly the entire tile space into the no-spill regime. *Per-tile peak rescue:* 8.04× at $(256, 128)$, where the four-mover intermediate exceeds the register file under without our optimization and the narrowed form does not under with our optimization. The 8.04× figure is the gain available at a fixed sub-optimal tile shape; the workload-level number above is a separate measurement. Across the 9-tile sweep: 11 tiles improve, 7 are neutral, 0 regress.

Mechanism at $(128, 256)$**:** Sinking the i8 truncation through the four data-movement operations means the doubled post-concatenation tile is materialized in i8 rather than fp32, i.e. 4× smaller per element. Under without our optimization the kernel sits at 64 registers per thread with 13.4 M local-memory spill loads; under with our optimization it reaches 203 registers per thread with *zero* spill, total instructions are halved (50.1 → 24.1 M) and mean execution time time falls from 9009 to 1233 μs. Global-memory loads and stores and the narrowing arithmetic instruction count are unchanged between OFF and ON — only the in-kernel staging traffic (spill traffic and the shared-memory barriers / load-shared-matrix / store-shared instructions used by Triton's lowering of the four movers) shrinks by a factor of four. Full NCU and instruction-mix tables appear in Appendix F. Torch-eager baseline runs 8522.5 μs; the best Triton with our optimization time (1214 μs) is 7.02× faster.

6.5 Pattern 3: Multi-consumer (two-way)

Kernel: `tt.cat(fp32, fp32)` → { `fptosi` → i8 store, `truncf` → fp16 store }. The pass's multi-consumer policy is exercised: dedup cache produces two narrowed cats (one i8, one fp16); the original fp32 cat is dead after rewrite.

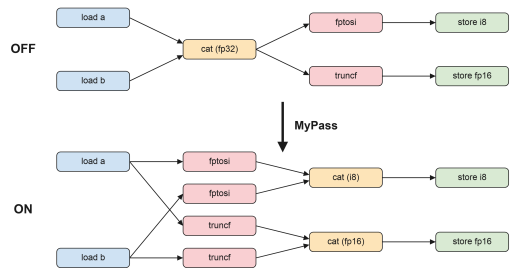

Figure 3: Operation flow for the Multi-consumer two-way diamond. With our optimization dedup cache duplicates the concatenation into two narrow chains, keyed on (OperationName, ResultType): one i8 chain for the `fptosi` consumer, one fp16 chain for the `truncf` consumer. At tile sizes where both narrow chains still exceed the register file the duplication is unhelpful.

Headline numbers: *Best-vs-best speedup:* **1.21**× (fastest without our optimization tile $(64, 128)$ at 2148 μs vs. fastest with our optimization tile $(64, 256)$ at 1769 μs). The dedup cache lets the post-concatenation intermediate be materialised in i8, the smaller of the two narrow types, which lifts the fastest configuration too. *Viable-tile expansion:* tiles within 20% of best without our optimization latency rise from **2/9** to **3/9**. *Per-tile peak rescue:* 1.28× at $(128, 128)$. *Worst tile:* 0.69× at $(128, 256)$ — the dedup cache produces two narrowed concatenations but at this tile both still exceed the register file, so the duplication is unhelpful. Across the 9-tile sweep: 6 tiles improve, 8 are neutral, four regress at large or asymmetric tiles. A register-pressure gate at the rewrite point would convert the four regressions into no-ops.

Mechanism, both directions: At the winning tile $(128, 128)$, without our optimization produces 1.5 M spill loads at 255 registers per thread and with our optimization eliminates all spill — the dedup cache emits two narrower concatenations and both fit. At the worst tile $(128, 256)$, without our optimization spills 17.7 M loads and with our optimization produces two narrowed concatenations that *both* spill (20.8 and 20.1 M in the duplicated chains): the duplication does not buy a fit, and the extra work costs wall-clock time. This bidirectional evidence is what makes the case for adding a register-pressure check at the rewrite point. Full per-tile NCU table in Appendix F.

6.6 Pattern 4: Transpose – Truncate

Kernel: `tl.trans` (fp32) → `.to(tl.float16)`. Single-mover, single-consumer counterpart of Pattern 1, with `tt.trans` instead of `tt.cat`.

Result: The pass fires at TTIR; the rewrite survives TTGIR (both differ between without our optimization and with our optimization). But the *LLIR, PTX, and cubin are bit-identical* across all 16 tile configurations: SHA-256 of `.llir`, `.ptx`, and `.cubin` match between OFF and ON in every cell. NCU hardware counters — registers, occupancy, throughput, duration, spill loads/stores — match within noise. nsys ratios across 16 configs span 0.992× to 1.002×.

Explanation: `tt.trans` permutes a tile in place without changing the live-element count. Truncating before vs. after produces isomorphic live-range graphs, so the register allocator makes the same decisions in both pass states. The Triton GPU lowering of `trans + truncf` collapses the IR difference during TTGIR → LLIR translation, generating the same instruction sequence regardless of source order. The same null-result mechanism applies to `tt.reshape` (Appendix A.1), where all 18 sweep configurations also report 1.00× and NCU spill counts match exactly. The TTIR-level rewrite is real, survives into TTGIR,

but is collapsed during TTGIR→LLIR lowering. From LLIR onward both builds are bit-identical. This is the strongest possible form of the no-op claim at the assembly level for a layout-only mover — i.e. a mover whose output has the same number of live elements as its input (here, the transpose merely permutes the axes of the tile), so narrowing the type before vs. after the mover produces isomorphic live-range graphs and yields no register-allocation difference.

Three further experiments are presented in the appendix: a multi-consumer fan-out on `tt.trans` (the transpose counterpart of Pattern 3); a concatenate-then-quantize chain (`tl.cat` → `fptosi`, the 4× bit-width-ratio sibling of Pattern 1); and a broadcast–truncate chain on a small vector. The transpose-diamond and the broadcast-on-cached-vector cases are the two experiments that most directly motivate adding a register-pressure or mover-shape check at the rewrite point. Full data and discussion appear in Appendix E.

6.7 Pattern 5: Multi-consumer (three-way)

Kernel: One `tt.cat(fp32, fp32)` fans out to three distinct reducer kinds: `arith.fptosi`→i8, `arith.truncf`→fp16, `tt.fp_to_fp`→fp8E5M2. The third path is the only one in the suite that exercises the `triton::FpToFpOp` clone branch and its `getRoundingAttr()` carry-through.

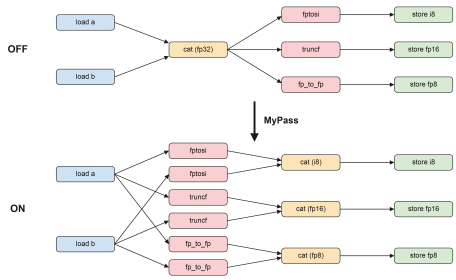

Figure 4: Operation flow for the Multi-consumer (three-way) diamond kernel (Pattern 5). The dedup cache distinguishes the three consumer kinds and emits three independent narrowed concatenation chains.

Structural verification: The post-pass TTIR contains exactly three rewritten cats (one per result type, no false sharing) and the original fp32 cat is erased; SHA-256 matches for all three outputs (i8, fp16, fp8E5M2) across 4 shapes.

Headline numbers: *Best-vs-best speedup:* **1.06**× (fastest without our optimization tile $(64, 128)$ at $2704\,\mu$s vs. fastest with our optimization tile $(64, 256)$ at $2547\,\mu$s). *Viable-tile expansion:* tiles within 20% of best without our optimization latency rise from **1/9** to **3/9**. *Per-tile peak rescue:* 1.58× at $(64, 256)$. *Worst tile:* 0.79× at $(256, 128)$ — with three cloned chains (one per consumer kind), the slowdown at non-fitting tiles scales approximately linearly with the chain count: two chains in Pattern 3 give 0.69× worst, three chains here give 0.79× worst at a different tile. The gain at the best tile is smaller than in Pattern 3 (1.06× vs. 1.21×) because the three distinct narrow types (fp16, i8, fp8E5M2) span a wider bit-width range, so the i8 chain must coexist with the fp16 and fp8 chains in registers. Across 36 configs (2 matrix sizes × 9 tiles): 8 wins, 6 neutral, 4 regressions. A register-pressure gate at the rewrite point would suppress the four regressions.

Mechanism, both directions: At the winning tile $(64, 256)$, spill loads and stores each drop 3.57 → 0.88 M and wall-clock time falls 4076 → $2547\,\mu$s. At the worst tile $(256, 128)$, all three cloned chains spill independently: spill loads rise 24.3 → 27.2 M, DRAM traffic increases by 1136 MB, and wall-clock time rises 14834 → $18788\,\mu$s. This is a second independent example of the cloned-chain pattern seen in Pattern 3, on a different kernel — the bidirectional behaviour is reproducible.

6.8 Complex DAG Patterns

The five experiments above cover linear chains and the canonical two- and three-way multi-consumer diamonds. We additionally exercised the pass on five DAG topologies that stress the dedup cache, the worklist, the bitwidth guard, and the gate that handles non-reducer users.

6.8.1 Dedup cache: hit, type-split, and kind-split

Three configurations share a one-producer / two-consumer structure but reach different paths through the dedup cache. **Type-split** is the canonical case already analysed in Pattern 3 (`tt.cat` feeding `fptosi`-i8 and `truncf`-fp16: two distinct *ResultTypes*, the cache yields two rewritten chains). The other two cases isolate the cache's behaviour in the limits where the two consumers share more than one key component:

Same-kind same-type fan-out (cache hit). Both consumers match on every key component. The textbook pattern (one cat consumed by two textually-identical `c.to(fp16)` calls) does *not* actually reach the hit path because Triton's frontend folds the two identical `arith.truncf` ops into one SSA value before `DowncastReorderOptimizer` runs. To drive the hit path we insert a distinct `reshape` between the cat and each truncf (different output shapes, frontend cannot merge); the pass propagates each truncf up through its reshape, and the two cloned truncfs meet at the cat with identical keys but distinct SSA values. The pass produces exactly one rewritten cat-on-fp16 shared between both branches.

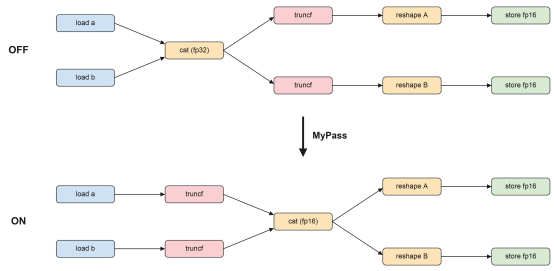

Figure 5: Cache-hit fan-out: both consumers' truncf-fp16 clones meet at the concatenation with identical (operation kind, result type) keys and the dedup cache returns one rewritten chain shared by both downstream reshapes.

Performance: 9 wins, 9 neutral, no regressions across 18 configs (2 matrix sizes × 9 tiles); best 1.25× at $(256, 64)$, $M = N = 8192$.

Mixed-kind same-type fan-out (cache split by operation kind). One fp32 `cat` feeds `arith.fptosi`→i8 and `arith.fptoui`→u8. On this Triton build both lower to the *same* signless MLIR result type `tensor<...xi8>`, so the two cache entries differ only in the operation-kind component of the key. The split is verified *necessary*, not merely policy-correct: `fptosi` and `fptoui` have different numeric semantics for the same fp32 input, so sharing one chain would emit wrong bits for one store.

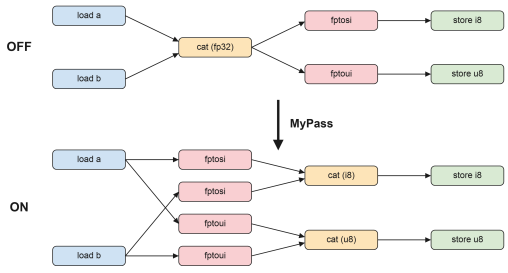

Figure 6: Mixed-kind cache split: the two consumers share the same result type but differ in operation kind, so the dedup cache produces two distinct rewritten chains. Sharing would emit wrong bits for one of the stores.

Performance: 4.72× at $(128, 256)$ is the largest two-consumer-diamond gain in the suite. Under without our optimization the kernel runs at 64 registers per thread and 95% occupancy with $\sim$ 31 M local-memory spill requests; with our optimization narrows both chains to i8 (4× each), fits in 255 registers at low occupancy, collapses spill to 1.8 M, and saves 3828 MB of DRAM traffic. The DRAM-bandwidth model attributes $\approx$ 100% of the per-tile delta to this saving (Section 7).

The two cases above together with Pattern 3 demonstrate that the cache key's two components (operation kind, result type) are both necessary: removing either would either merge chains that must remain distinct (correctness break) or split chains that could safely share (missed optimisation).

6.8.2 Multi-level diamond (cache and worklist together)

Combines the dedup cache and the worklist on one DAG: a `tt.cat` fans out into two branches, each with further movers (one branch is just a `reshape`, the other is `trans`+`reshape`) before terminating in `truncf`-fp16. The cat is not initially eligible — its direct users are reshapes, not reducers — so it can only be rewritten after the worklist propagates the reducers up and re-queues it. The end state: one cat-on-fp16; the two cloned truncfs meet at the cat with identical fp16 keys, a cache hit driven by worklist propagation rather than by source duplication.

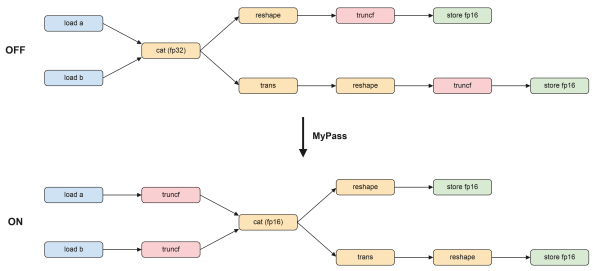

Figure 7: Multi-level diamond: three or more worklist pops are required before the upstream concatenation becomes eligible. The two truncate casts propagate up through their downstream movers and meet at the concatenation with identical fp16 keys, yielding a cache hit by propagation rather than by source duplication.

Performance is numerically very close to the cache-hit case above (best 1.24× at $(256, 64)$, $M = N = 8192$; same 9/9/0 wins-neutral-regress distribution across 18 configs), confirming the multi-level path reaches the same rewritten structure.

6.8.3 Nested concatenations (cat-of-cats)

Three `tt.cat` ops chained as `cat(a, b)` and `cat(c, d)` feeding `cat(ab, cd)`, terminated by one `truncf`-fp16. The initial walk seeds only the outer cat. The worklist propagates the rewrite up through both inner cats in a single `runOnce` sweep (the outer 16-iteration fixed-point loop is never engaged in practice).

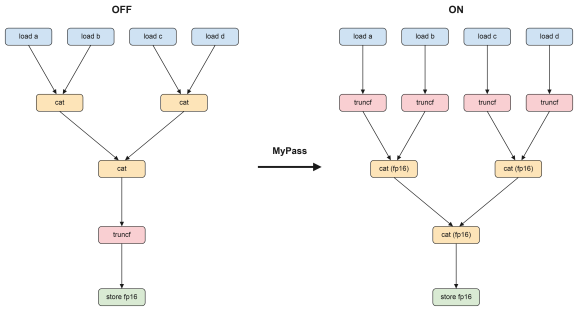

Figure 8: Nested concatenations: one runOnce sweep propagates four cloned truncate casts onto each of the four loaded operands, with all three nested concatenations rewritten to operate on fp16.

5.69× at $(64, 256)$ is the largest single-tile gain in this group of complex DAG patterns; the kernel also rescues at $(128, 128)$ (5.52×) and $(256, 64)$ (5.13×), so the rescue is not a single freak tile. Pattern 9 is also where the new 9-tile data shifts the workload-level B-v-B from 1.07× to 1.17×, because the newly-measured $(64, 128)$ ON tile turns out to be the fastest with our optimization tile in the kernel. Four worst-tile entries (0.89–0.94×) appear at non-square large tiles: with four cloned chains the per-chain register pressure swings sharply with tile shape. The same chain-count trend recurs through the suite — two cloned chains in Pattern 3 (0.69× worst), three in Pattern 5 (0.79× worst), four here (0.89× worst) — and a register-pressure check at the rewrite point would suppress all of these worst-tile entries.

6.8.4 Per-operand bit-width guard (negative control)

Tests the guard that skips a consumer unless every producer operand's element bit-width is strictly greater than the consumer's destination. The kernel has two paths in one module: `load fp16` → `reshape` → `fptosi` → `i16` (guard *fails*: 16 = 16) and `load fp32` → `reshape` → `fptosi` → `i16` (guard *passes*). with our optimization leaves path A unchanged and rewrites path B. Perf: 10/10 neutral, assembly opcode histograms identical. A verified no-op when the pass should skip.

6.8.5 Shared-upstream Y-shape (CSE interaction)

One shared fp32 load routes into two independent mover-+-truncate chains (one via `tt.trans`, one via `tt.reshape`). The pass rewrites each chain independently, so the load ends up with two cloned truncate consumers with identical operands and result types. Downstream MLIR canonicalize and CSE deduplicate the redundant clones: TTGIR drops from two `arith.truncf` ops to one and the assembly is byte-identical between OFF and ON (128 F2F casts in both). 10/10 neutral with byte-identical assembly. The pass can safely emit redundant clones for Y-shape upstreams because the pipeline composition cleans them up; no special-case de-duplication logic is needed in `DowncastReorderOptimizer`.

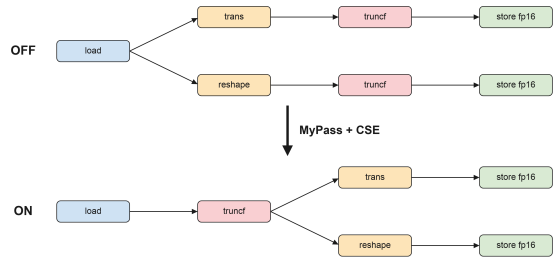

Figure 9: Y-shape with downstream CSE: `DowncastReorderOptimizer` emits two identical truncate casts on the shared load; downstream common-subexpression elimination merges them back into one, so the final assembly is byte-identical to without our optimization.

6.8.6 Asymmetric diamond (gate behaviour)

Kernel: `tt.trans` (fp32) fans out into one `truncf`-to-fp16 store and one direct fp32 store. The trans output has one reducer user and one non-reducer user. This directly exercises the all-consumers-reducible bailout (guard 5).

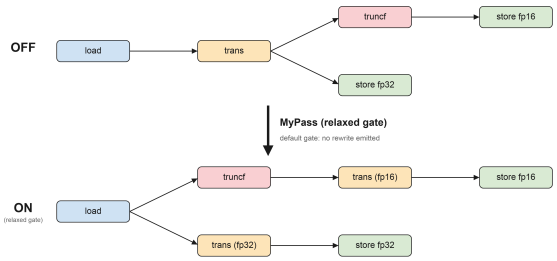

Figure 10: Asymmetric diamond on `tt.trans`. Under the default strict gate, guard 5 bails because one of the two consumers is not a reducer, and the without our optimization and with our optimization IR are byte-identical. Under a relaxed-gate rebuild of the pass, the rewrite fires at zero wall-clock cost because duplicating a permutation does not enlarge the live set.

Strict gate (default). The bailout fires; without our optimization and with our optimization produce byte-identical IR. All 18 configurations report 1.00×.

Relaxed gate (pass patched). The bailout's `return {}` on a non-reducer user is replaced by a `continue` that skips the non-reducer and keeps the size-reducing user. The diamond rewrite fires, and the post-pass IR confirms two coexisting `tt.trans` ops (original fp32, cloned fp16). Wall-clock: all 18 configurations remain at 1.00× (without our optimization/with our optimization ratios 0.999–1.004). NCU at $(256, 256)$ shows the rewrite *reduces* spill traffic (loads 11.3 → 9.5 M, stores 10.0 → 6.2 M) because the fp16 narrow chain spills fewer bytes, but the kernel is DRAM-bandwidth-bound (71–76% DRAM utilisation) and the reduced local-memory traffic does not surface as a wall-clock change. The asymmetric `trans` case is a clean refinement target for a mover-aware relaxation of guard 5 — the rewrite is safe and cheap, but the default gate refuses it.

6.9 Performance Summary Across the Suite

The table below separates the two performance numbers each experiment produces. **Best-vs-best** (B-v-B) is the fastest-without our optimization vs fastest-with our optimization ratio at $M = N = 8192$ — the *workload speedup*. **Per-tile peak rescue** is the largest improved(in terms of execution time) single-tile without our optimization/with our optimization ratio — the value of the pass at a fixed sub-optimal tile shape. **Viable tiles** is the count of tile shapes within 20% of the best without our optimization latency.

Reading the table: the linear-chain experiments (Pattern 1, its Pattern 12 sibling, and Pattern 2) have best-vs-best of 1.01–1.04× but per-tile peak rescues of 3.90–8.04×, and they roughly double the viable-tile set. The multi-consumer diamond family (Patterns 3, 5, 6, 7, and 8) produces workload-level gains in the 1.06–1.24× range, with Pattern 7's cache split the largest (1.24× best-vs-best, 4.72× peak rescue) and Pattern 3 at 1.21×. Pattern 9 (nested concatenations) sits at 1.17× best-vs-best with the largest peak rescue in the group (5.69×). These gains come from the dedup cache changing structural register usage at the fastest tile, paired with worst-tile entries (0.69–0.89×) on shapes where every cloned narrow chain still exceeds the register file. The layout-only chains (Pattern 4 and the Patterns 10 and 11 controls) report 1.00× at every tile. Aggregating across 220 benchmarked configurations: 73 wins ($\geq 1.10\times$), 129 neutral, and 18 worst-tile entries that all sit at non-square or extreme-aspect tiles where every cloned narrow chain still exceeds the register file.

Pattern	Kernel	B-v-B	Viable Tiles (Without Optimization→With Optimization)	Peak rescue	Worst Performance	Triton/Torch
Pattern 1	Concatenate – Truncate	1.04×	$3 \to 6$	3.90×	1.00×	2.40×
Pattern 2	Deep Mover Chain	1.01×	$5 \to 8$	8.04×	1.00×	7.02×
Pattern 3	Multi-consumer	1.21×	$2 \to 3$	1.28×	0.69×	3.01×
Pattern 4	Transpose – Truncate	1.00×	$3 \to 3$	1.00×	1.00×	3.74×
Pattern 5	Multi-consumer (3-way)	1.06×	$1 \to 3$	1.58×	0.79×	2.35×
Complex DAG patterns (Section 6.8):						
Pattern 6	cache hit (same-kind same-type)	1.09×	$3 \to 3$	1.25×	0.98×	2.44×
Pattern 7	cache split (mixed-kind)	1.24×	$2 \to 4$	4.72×	0.99×	3.11×
Pattern 8	multi-level diamond	1.09×	$3 \to 3$	1.24×	0.99×	4.92×
Pattern 9	nested concatenations	1.17×	$2 \to 6$	5.69×	0.89×	3.38×
Pattern 10	bit-width guard (no-op)	1.00×	$2 \to 2$	1.00×	1.00×	1.08×
Pattern 11	Y-shape with CSE	1.00×	$4 \to 4$	1.00×	1.00×	1.38×
Appendix experiment (Section E):						
Pattern 12	Concatenate – Quantize	1.02×	$6 \to 8$	7.09×	1.00×	2.63×

Table 1: Best-vs-best (workload speedup), per-tile peak rescue, worst performing for a tile size, and best Triton with our optimization versus an unfused torch-eager baseline, $M = N = 8192$. The B-v-B and Peak rescue columns measure different things; see the Discussion section. "Viable" is the count of tile shapes (out of 9, over the full $\{64, 128, 256\}^2$ grid) within 20% of the best without our optimization latency. The Triton/Torch column gives the speedup of the best Triton with our optimization tile over an unfused PyTorch chain that materialises every intermediate.

Figure 11: Per-tile speedup heatmap at $M = N = 8192$. One 3×3 grid per experiment; rows are `BLOCK_M` and columns are `BLOCK_N` (both $\in \{64, 128, 256\}$). Each cell is the (without our optimization)/(with our optimization) execution time ratio at that tile, coloured on the legend at the right. Heatmaps make the spatial structure of the rescue visible: Pattern 1, Pattern 2, Pattern 7, Pattern 9, and Pattern 12 have two adjacent big-rescue cells at non-square shapes (the spill-cliff diagonal); Patterns 3 and 5 show wins on small tiles and worst-tile entries on large asymmetric ones; Pattern 4, Patterns 10 and 11 are uniformly 1.00×.

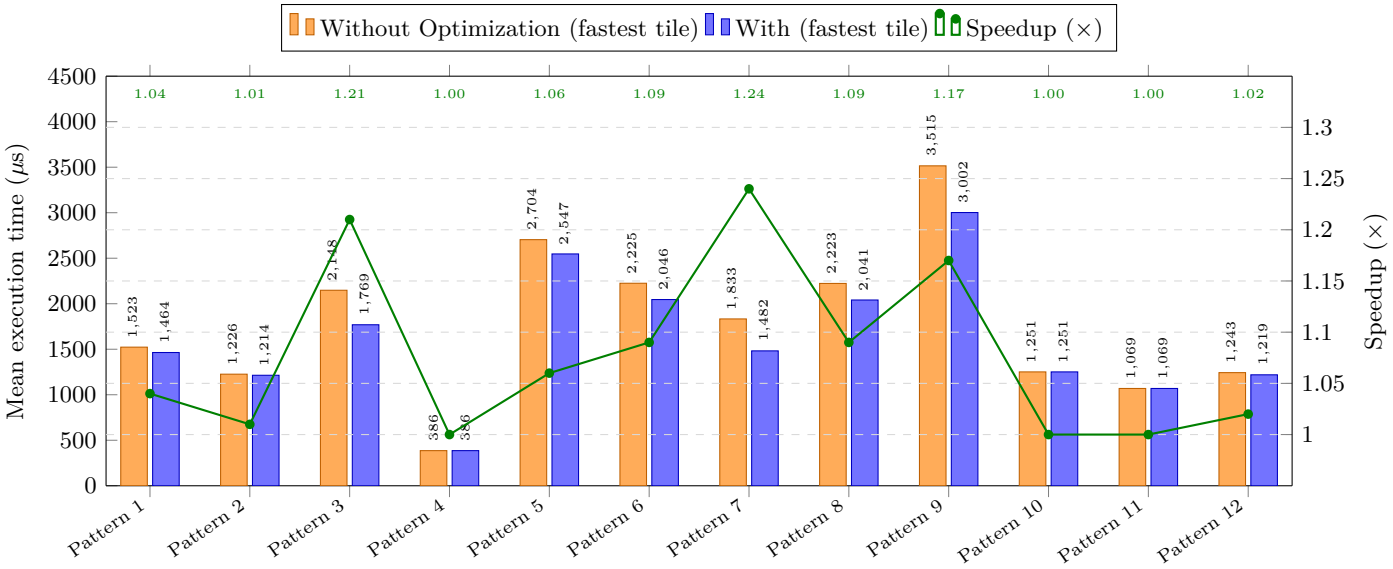

Figure 12: Real execution time at the best tile per pattern, $M = N = 8192$. Orange = fastest without our optimization tile, blue = fastest with our optimization tile (mean execution time in microseconds, linear scale). The orange→blue gap is the workload speedup that an autotuner with unlimited budget would see; it is small on linear chains (Patterns 1, 2, 12) because both states already fit a small tile, and larger on the multi-consumer diamond family (Patterns 3, 5, 6, 7, 8). Patterns 4, 10, and 11 have without our optimization and with our optimization bars at exactly the same height.

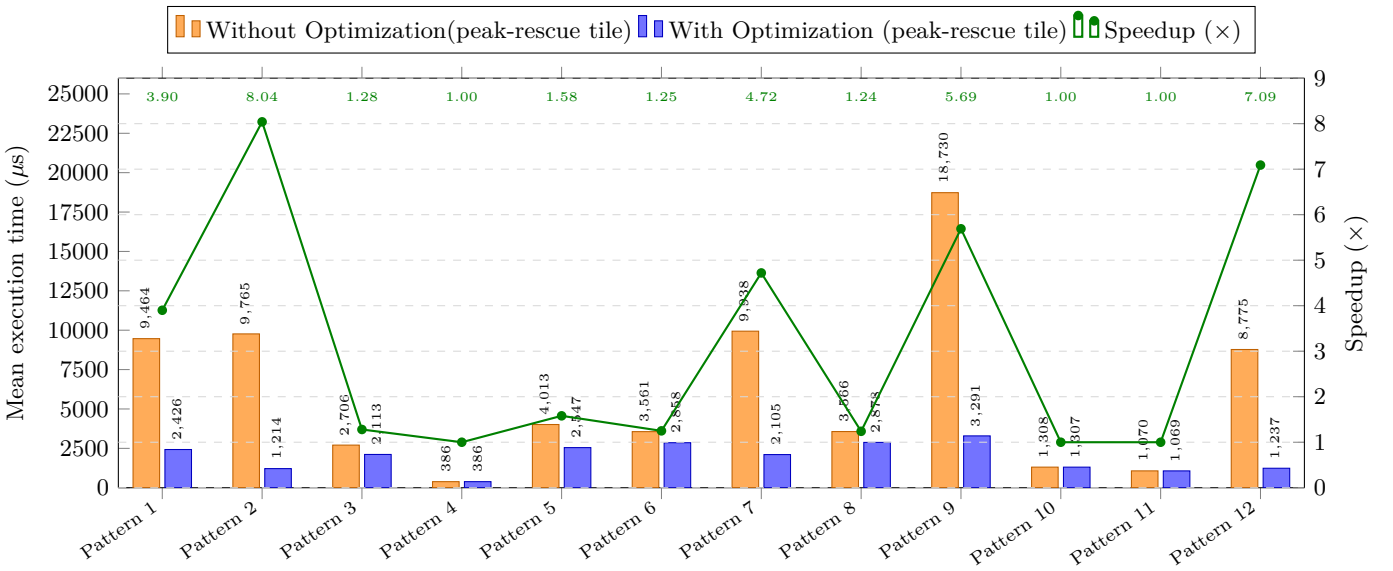

Figure 13: Real execution time at the peak-rescue tile per pattern, $M = N = 8192$. The peak-rescue tile is the tile that gives the largest single-tile (without our optimization)/(with our optimization) execution time ratio; this is the gain available to a developer or autotuner committed to that tile. Orange = without our optimization, blue = with our optimization (mean execution time in microseconds, linear scale). The largest absolute gaps appear on Pattern 9 (nested cats, 18,730 → 3,291 μs), Pattern 7 (cache split, 9,938 → 2,105 μs), Pattern 2 (deep mover chain, 9,765 → 1,214 μs), Pattern 12 (Concatenate – Quantize, 8,775 → 1,237 μs), and Pattern 1 (Concatenate – Truncate, 9,464 → 2,426 μs).

7 Causal Verification: The DRAM-Bandwidth Model

The experiments establish a correlation: when the pass fires, spill traffic drops and wall-clock time drops. This section strengthens that into a causal claim by applying a single quantitative model across 11 configurations from 6 kernels.

7.1 Model and Protocol

The predicted wall-time change is

$$\Delta t_{\text{pred}} = \frac{\text{DRAM bytes}_{\text{OFF}} - \text{DRAM bytes}_{\text{ON}}}{\text{DRAM rate}_{\text{ON}}}$$

and the fraction of the measured change explained by the model is $|\Delta t_{\text{pred}}|/|\Delta t_{\text{wall}}|$. The single-variable model is justified by the chain of reasoning of Section 4: when the wide intermediate exceeds the per-SM register file and the narrow one does not, the kernel exits the spill regime; overflowed spill traffic that bypasses the per-SM L1 reaches DRAM; on bandwidth-bound kernels (every configuration in this report has at least 50% DRAM throughput utilisation), removing those bytes removes wall-clock time at the DRAM rate.

For each (kernel, tile) we ran a seven-step protocol: **(1)** bit-exact SHA-256 correctness on every output; **(2)** assembly-level algorithmic invariance — the global-memory load, global-memory store, and narrowing-cast instruction counts are unchanged between Without Optimization Run and With Optimization Run; **(3)** locked launch configuration (grid, warps, stages, registers per thread, waves) cross-checked between Triton metadata and NCU; **(4)** spill bookkeeping — compiler-side spill-slot count versus NCU's runtime local-memory request counters; **(5)** DRAM accounting — compare Δt_{pred} to Δt_{wall}; **(6)** instruction-count accounting — the per-warp executed instruction count delta versus the prediction obtained from the per-opcode assembly diff; **(7)** warp-stall-reason shift — on a win the local-memory-pipeline-saturation stall, the barrier stall, and the short-scoreboard stall all fall, while the long-scoreboard stall rises because warps now expose unavoidable global-memory latency rather than the artificial load/store-unit contention produced by spill traffic.

7.2 Results

The model accounts for 66 to roughly 100% of the wall-time delta on 9 winning configurations and 58 to 70% of the slowdown on two regressing configurations. The two regressions appear on two distinct multi-consumer diamonds (Patterns 3 and 5), so the bidirectional behaviour is not a single-kernel coincidence.

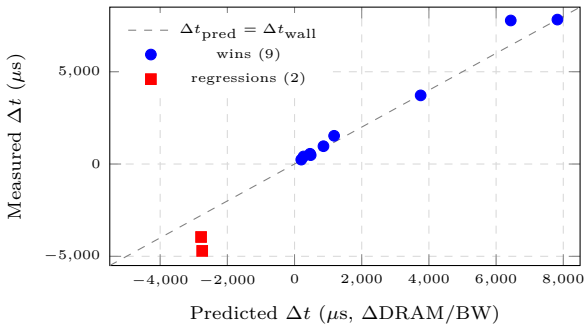

Figure 14: DRAM-bandwidth model: predicted vs. measured wall-time delta across 11 verified configurations from 6 kernels. The dashed line is $\Delta t_{\text{pred}} = \Delta t_{\text{wall}}$ (perfect prediction). Points above the line (mostly the wins) have a residual explained by coupled non-DRAM effects (barrier reduction, shared-memory traffic reduction). The two red squares are the regression cases; the model predicts the direction and most of the magnitude in both directions.

The residual (0 to 42%, smallest where the model fits tightest) is attributable to non-DRAM effects that move with the same rewrite: barrier reduction, shared-memory traffic reduction, instruction-count change, and the additional permute instructions inserted by the narrower lowering. These cannot be decomposed by direct measurement because the pass produces all four effects as a single atomic IR rewrite that cannot be toggled independently. The cleanest single datapoint is Pattern 2 at $(128, 128)$, where the assembly-only prediction of per-warp instruction delta (95) matches the measured value (94) to within 1%, leaving essentially no room for a hidden contribution.

7.3 Stall-Reason Shift Across the Suite

The pass's effect on warp-stall reasons is consistent across every verified configuration. The local-memory-pipeline-saturation stall drops sharply on wins and rises on regressions; the barrier stall falls in proportion to the eliminated shared-memory staging; and the long-scoreboard stall (warps blocked on long-latency global-memory dependencies) rises on wins because warps now expose unavoidable global-memory latency rather than the artificial load/store-unit contention produced by spill stores.

Pattern 2's mild-win row shows the cleanest signature: the local-memory stall drops because spill traffic falls, the long-scoreboard stall more than doubles because warps now wait on real global memory rather than on spill, and the barrier stall falls because fewer shared-memory staging boundaries remain. On the regressing configurations the directions invert: the local-memory stall rises because the duplicated chains spill more, not less.

7.4 L1-Absorption Gradient

The magnitude of a win depends on the fraction of spill bytes absorbed by the per-SM L1 cache ($\sim$ 64 KB unified L1 / shared on the 2080 Ti). At Pattern 1 tile $(128, 128)$, L1 absorbs about 60% of the spill bytes and the speedup is 1.28×. At Pattern 2 tile $(128, 128)$, L1 absorbs 39% and the speedup is 1.19×. At Pattern 2 tile $(128, 256)$, L1 absorbs only $\sim 1\%$ (the working set is too large) and the speedup is 7.31×. The larger the spill working set relative to L1 capacity, the more spill bytes reach DRAM and the larger the bandwidth gain from removing them.

8 Discussion

8.1 One mechanism, both directions

Every measured per-tile speedup and every worst-tile entry in this report is explained by one mechanism. The pass narrows the element type of the largest live intermediate that a mover materialises. When the wide (pre-rewrite) intermediate exceeds the per-SM register file and the narrow (post-rewrite) one does not, the kernel crosses out of the spill regime: the register allocator stops evicting live values to local memory; the spilled traffic that overflowed L1 to DRAM vanishes; and on these bandwidth-bound kernels (every configuration in the report is at least 50% DRAM throughput) removing those bytes removes wall-clock time at the DRAM rate.

The effect is discontinuous in tile size because spilling is a phase transition: a single tile-size step crossing the register-fit boundary swings wall-clock by a factor of 5 to 8 while the algorithmic work only doubles. The pass relocates this boundary outward by the bit-width factor (2× for fp16, 4× for i8); whether a given tile lands on the safe side depends on the pre-rewrite working set, the post-rewrite chain count, and the per-SM register budget.

Per-tile peak rescue and best-vs-best measure different things: The per-tile peak rescue numbers (3.90× on Pattern 1, 7.09× on Pattern 12, 8.04× on Pattern 2) measure the gain available at a fixed tile shape — what the pass saves a developer or autotuner committed to that tile. The best-vs-best number compares the fastest configuration in each pass state and is small on linear chains (1.01–1.04×) because some small tile already fits in registers in both pass states. The pass's impact on linear chains is the *viable-tile set expansion*(the count of tile shapes within 20% of the best without our optimization latency): it roughly doubles the set of tile shapes that fit in registers — $3/9 \rightarrow 6/9$ on Pattern 1, $5/9 \rightarrow 8/9$ on Pattern 2, $6/9 \rightarrow 8/9$ on the Pattern 12. A wider viable-tile set is useful when the tile is constrained externally or when autotune is time-limited.

Multi-consumer diamonds are the exception, in both directions: The diamond family (Patterns 3 at 1.21×, 5 at 1.06×, 6 at 1.09×, 7 at 1.24×, and 8 at 1.09× best-vs-best) show meaningful workload-level gain because the dedup cache changes the live-range graph at the fastest tile too: the post-concatenation intermediate now lives at multiple narrow types instead of one wide type. The same duplication produces the worst-tile entries: on tiles where every cloned chain still exceeds the register file, the duplication is strictly a cost. The worst-tile speedup tracks the chain count k approximately linearly: $k = 2$ gives 0.69× (Pattern 3), $k = 3$ gives 0.79× (Pattern 5), $k = 4$ gives 0.89× (Section 6.8, nested concatenations). The DRAM-bandwidth model predicts the slowdown magnitudes (-1146 MB and -1136 MB added DRAM at the worst tiles of Patterns 3 and 5) to within 58 to 70%; the residual is the extra dynamic instructions from running the cloned chains' permute/integer-multiply-add machinery in parallel.

Bandwidth-roof sanity check: At Pattern 2 tile $(128, 256)$ the ON kernel achieves 542 GB/s, which is 88% of the 2080 Ti's practical bandwidth ceiling ($\sim$ 550 GB/s) and about 99% of a streaming copy-and-cast benchmark on the same device. At this point there is essentially no headroom left for an alternative compiler trick; the extra time in run without our optimization at this tile is spent on the spill traffic, barrier instructions, and shared-memory staging that the pass removes. The attribution is therefore not statistical — it is bounded above by the device.

9 Limitations and Future Work

Limitations: (i) *Intra-kernel scope:* the rewrite is restricted to a single Triton kernel; cross-kernel producer-consumer patterns require manual fusion. (ii) *Element-wise reducers only:* shape-reducing operations (`tt.reduce` sum/max over non-trivial dimensions) need attribute remapping (like XLA's `HandleReduce`) and are not handled in our optimization. (iii) *No register-pressure check:* the pass fires whenever its local guards pass, which is too aggressive on multi-consumer diamonds at tile shapes where every cloned narrow chain still exceeds the register file.

Future work: A lightweight register-pressure estimate at the rewrite point would convert the diamond regressions into no-ops (a simple sum of operand tile sizes scaled by destination bit-width catches the regressions we observe). Extending the pass to broadcast-amplifier patterns at workload sizes that actually materialise the broadcasted tile, and to shape-altering reductions (`tt.reduce`), closes the largest remaining coverage gaps. Cross-architecture validation on A100/H100 (larger register files, different L1 capacities) will shift the register-fit boundary; the mechanism is architecture-agnostic but the specific speedup magnitudes are not.

10 Conclusion

We designed and implemented `DowncastReorderOptimizer`, a Triton compiler optimization that sinks size-reducing operations upstream of data-movement operations via worklist-driven fixed-point iteration over the TTIR. The pass handles six movers and six element size-reducers, supports multi-consumer diamond fan-outs through an `(OperationName, ResultType)` dedup cache, and is value-preserving on every DAG topology we evaluate (bit-exact SHA-256 verified across 14 kernels.

Across the microbenchmark suite the pass produces two distinct performance effects, reported separately because they answer different questions. The **best-vs-best workload speedup** (fastest without our optimization performance vs. fastest with our optimization performance) is 1.01–1.04× on the linear mover chains and reaches 1.24× on the multi-consumer diamond family (best at Pattern 7's cache split; the canonical two-way diamond Pattern 3 reaches 1.21×). The **per-tile peak rescue** (same-tile without our optimization/with our optimization ratio) is much larger — 3.90× on Pattern 1, 7.09× on Pattern 12, 8.04× on Pattern 2, 4.72× on Pattern 7, and 5.69× on Pattern 9 — but measures the gain at a fixed sub-optimal tile shape, not how much faster the workload runs at the best tile. A small number of worst-tile entries appear on multi-consumer diamonds at shapes where every cloned narrow chain still exceeds the register file (0.69× on Pattern 3, 0.79× on Pattern 5).

On linear chains the pass roughly doubles the *viable-tile fraction* — the set of tile shapes that fit in registers — without changing the fastest tile. On multi-consumer diamonds the pass changes structural register usage and produces a workload- level gain at the best tile, with worst-tile entries on configurations where the duplication does not buy a fit.

A single quantitative model $\Delta t_{\text{pred}} = (\Delta\text{DRAM bytes})/(\text{ON DRAM rate})$ accounts for 66% to roughly 100% of the per-tile wall-time delta on 9 winning configurations and 58–70% on 2 regressing configurations across 11 configurations from 6 experiments — in both directions of the mechanism (DRAM saved ⇒ speedup; DRAM added ⇒ slowdown).

The single actionable conclusion is that the all-consumers-reducible and per-operand bit-width guards keep the pass correct on every DAG, and adding a register-pressure check at the rewrite point would convert the diamond worst-tile entries into no-ops while preserving the diamond wins. This is the natural next step for the pass.

References

[1] Philippe Tillet, H. T. Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, MAPL 2019, page 10–19, New York, NY, USA, 2019. Association for Computing Machinery.

[2] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: an insightful visual performance model for multicore architectures. *Commun. ACM*, 52(4):65–76, April 2009.

[3] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. Mlir: A compiler infrastructure for the end of moore's law, 2020.

[4] P. Griffiths Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. Access path selection in a relational database management system. In *Proceedings of the 1979 ACM SIGMOD*

International Conference on Management of Data, SIGMOD '79, page 23–34, New York, NY, USA, 1979. Association for Computing Machinery.

[5] Philip Wadler. Deforestation: transforming programs to eliminate trees. *Theoretical Computer Science*, 73(2):231–248, 1990.

[6] Duncan Coutts, Roman Leshchinskiy, and Don Stewart. Stream fusion: from lists to streams to nothing at all. In *Proceedings of the 12th ACM SIGPLAN International Conference on Functional Programming*, ICFP '07, page 315–326, New York, NY, USA, 2007. Association for Computing Machinery.

[7] Zhihao Jia, Oded Padon, James Thomas, Todd Warszawski, Matei Zaharia, and Alex Aiken. Taso: optimizing deep learning computation with automatic generation of graph substitutions. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, SOSP '19, page 47–62, New York, NY, USA, 2019. Association for Computing Machinery.

[8] Haojie Wang, Jidong Zhai, Mingyu Gao, Zixuan Ma, Shizhi Tang, Liyan Zheng, Yuanzhi Li, Kaiyuan Rong, Yuanyong Chen, and Zhihao Jia. PET: Optimizing tensor programs with partially equivalent transformations and automated corrections. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*, pages 37–54. USENIX Association, July 2021.

[9] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference, 2017.

Appendix

A Additional Null-Result Experiments

A.1 `reshape` + quant (fp32 → i8)

Kernel: Single `tl.reshape` followed by `fptosi` to i8. All 18 sweep configurations report 1.00× to two decimal places.

Both pass states spill exactly 405,504 load/store requests at $(256, 256)$ — the same values are spilled in both because reshape does not gate any live-range boundary that element-width reduction could relieve. Mechanism identical to Pattern 4 (`trans`+`truncf`): layout-only movers do not create a spill-fit boundary, so the pass has no surface to act on.

Torch baseline: `A.reshape(M*N).to(torch.int8).reshape(M,N)` runs at 640.9 μs at $M = N = 8192$; best Triton with our optimization (671.2 μs) is 0.95× — the only experiment where Triton is no faster than torch eager. Reason: the chain reduces to a single cast; there is no intermediate to fuse, no register-pressure boundary to cross, no materialization to avoid.

A.2 `join` + quant (fp32 → i8)

Kernel: `tt.join(fp32, fp32)` (stack along new innermost axis) → `fptosi` → i8 store.

Best 1.12× at $(256, 256)$, $M = N = 4096$; 17 of 18 configurations at 1.00×. At $(256, 256)$ NCU shows without our optimization spills 11.08 M LD / 11.41 M ST, with our optimization 9.99 M LD / 10.08 M ST (a $\sim 11\%$ reduction); `lg_throttle` drops 104,452 → 47,881. The rewrite cannot eliminate spills here because joining two 256×256 tiles produces a working set exceeding the register file regardless of element width — only the volume of spilled values shrinks, and the speedup is correspondingly partial.

Torch baseline: `torch.stack([A,B], dim=-1).to(int8)` runs at 7107.1 μs at $M = N = 8192$; best Triton with our optimization (1216.1 μs) is 5.84× faster — the second-largest fused-vs-eager gap in the suite, driven primarily by removing the fp32 join intermediate materialization, with the pass contributing only at $(256, 256)$.

A.3 Multi-consumer trans (`multi_consumer_trans_trunc_2d`)

Kernel: `tt.trans` (fp32) → { `fptosi` → i8 store, `truncf` → fp16 store }. The `tt.trans` counterpart of Pattern 3's `tt.cat` diamond.

Result: The pass fires correctly — TTIR confirms two independent rewritten chains — but the sweep produces zero wins, 16 neutral configurations, and two regressions (down to 0.71× at $(256, 128)$, $M = N = 8192$).

At $(256, 128)$ without our optimization spills modestly with the 255-register budget; with our optimization duplicates the `trans` into two chains, the second chain's working set spills, and total spill loads rise 2.4×, stores 2.6×. The pass on a non-size-changing mover can only cost (chain duplication), never pay (no register-fit boundary to cross). This isolates the conclusion of Pattern 4 (single-consumer `trans`) under multi-consumer fan-out: the dedup cache is correct structural machinery, but a speedup requires a size-changing mover.

B Mover Semantics Summary

Of the six movers, `tt.broadcast` is the only one with strictly more output elements than input; pushing a cast above it reduces both the cast count and the materialized tile size by the broadcast factor. `tt.cat` and `tt.join` preserve total element count but materialize the largest live intermediate in their chains at twice the per-operand tile size, so they create the spill-fit boundary the pass crosses (mechanism 3). `tt.trans`, `tt.reshape`, and `tt.expand_dims` are layout-only and create no boundary — their experiments produce verified SASS-level no-ops.

C Torch Eager Baseline Summary

The largest fused-vs-eager gaps appear on chains that the pass *also* optimizes well (deep chain 7.0×, join 5.8×); the smallest gap (reshape, 0.95×) is on the chain that reduces to a single cast with no fusable intermediate. The pass and fusion gains are complementary: fusion removes the per-op materialization across

kernel boundaries; the pass removes the in-kernel spill-induced materialization across the spill-fit boundary.

D Reproducibility

Hardware/software: NVIDIA RTX 2080 Ti (Turing, 68 SMs, CC 7.5, 11 GB GDDR6); driver 535.104.12, CUDA 12.2; Triton 3.6.0 built from source with `DowncastReorderOptimizer` added to the Triton transforms directory; PyTorch 2.9.1+cu128; `nsys 2023.2.3`; `ncu 2023.2.2` (run under `sudo` for perf-counter access).

Pass toggle: The pass is toggled by commenting or uncommenting its registration in the Triton Python pass-pipeline setup; the Triton cache is wiped between toggles.

Determinism and verification: A pinned seed (`0xBEEFCAFE`) is used for input generation; each kernel's `run_verify.sh` runs the "without optimization" and "with optimization" variants under separate `TRITON_CACHE_DIR`s, hashes every output tensor with SHA-256, and asserts a bit-exact match. Compilation artifacts (`.ttir`, `.ttgir`, `.llir`, `.ptx`, `.cubin`) are also hashed; for `trans+trunc` the `.llir`, `.ptx`, and `.cubin` hashes match between not optimized and optimized, which is the strongest possible form of the SASS-level no-op claim.

Sweep scripts: Each experiment directory contains `run_nsys_sweep_2d.sh` (timing sweep, 9 or 36 configs per matrix size $\times 2$ pass states), `run_ncu_sweep_2d.sh` (hardware-counter drill-down on a small set of representative tiles), and an `aggregate_results.py` that produces the human-readable report tables.

E Additional Experiments (multi-consumer transpose, concatenate–quantize, broadcast)

The following three experiments were referenced from Section 6 and collected here to keep the main-body experiment list focused on the configurations that exercise distinct mechanisms.

E.1 Multi-consumer transpose

Kernel: `tt.trans` (fp32) $\rightarrow$ { `fptosi` $\rightarrow$ i8 store, `truncf` $\rightarrow$ fp16 store }: the `tt.trans` counterpart of Pattern 3's `tt.cat` diamond. The pass's multi-consumer policy fires (the dedup cache produces two narrowed transpose chains), but the underlying mover is layout-only.

Result: Zero wins across 18 configurations, 16 neutral, two regressions: $0.71\times$ at $(256, 128)$ for $M = N = 8192$ and $0.73\times$ at the same tile for $M = N = 4096$. At the 255-register ceiling, without our optimization produces a modest ~ 900 K spill loads; with our optimization duplicates the transpose into two chains, the second chain's working set spills, total spill loads rise by a factor of 2.4, and instructions rise by a factor of 1.07. The single-consumer transpose (Pattern 4) and this multi-consumer transpose together isolate a sharp result: the dedup cache and worklist are correct structural machinery, but the rewrite produces a speedup only when the mover changes the live-element count. A transpose permutes a tile in place, leaves the live-range graph isomorphic, and offers no register-fit boundary to cross; only the duplication remains. A mover-shape check at the rewrite point would suppress this case.

E.2 Concatenate – Quantize

Kernel: Sibling of Pattern 1 with i8 instead of fp16 output: two fp32 tiles, flatten, `tt.cat`, `fptosi` to i8, reshape, store. The bit-width ratio is $4\times$ (fp32 to i8) instead of $2\times$ (fp32 to fp16), so the rewrite relocates the register-fit boundary by a factor of 4 rather than 2.

Table 6: Per-tile speedup ($without our optimization / with our optimization$), $M = N = 8192$.

	64	128	256
m=64	1.00	1.01	1.05
m=128	1.01	1.10	**7.09**
m=256	1.07	6.90	1.29

Headline numbers: *Best-vs-best speedup:* **1.02**$\times$ (fastest without our optimization tile $(64, 128)$ at $1248\,\mu$s vs. fastest with our optimization tile $(64, 256)$ at $1219\,\mu$s). *Viable-tile expansion:* tiles within 20% of best without our optimization latency rise from **6**/**9** to **8**/**9**. *Per-tile peak rescue:* $7.09\times$ at $(128, 256)$, larger than Pattern 1's $3.90\times$ because the bit-width ratio is $4\times$ instead of $2\times$. Torch-eager baseline `torch.cat([A,B], dim=1).to(torch.int8)` runs at $3211\,\mu$s; best Triton with our optimization ($1219\,\mu$s) is $2.63\times$ faster.

E.3 Broadcast − Truncate

Kernel: A small fp32 vector (size N) is broadcast along a new axis to a $(\text{BLOCK}_M, \text{BLOCK}_N)$ tile and truncated to fp16. This is the only kernel in the suite where the mover satisfies $n_{\text{out}} > n_{\text{in}}$: the rewrite reduces the cast count by the broadcast factor (in addition to narrowing the bytes the broadcast moves). At $\text{BLOCK}_M = 256$ the ON variant casts 256× fewer elements; this is the theoretical maximum lever the pass exposes on a broadcast.

Result: The broadcast kernel is uniformly compute-light and memory-light (the broadcast itself is a layout-level replication with no global-memory write amplification) and is not memory-bound at the sweep sizes used here. The per-tile sweep at $M = N = 8192$ shows a uniform 1.00× across all 9 tiles. The mechanism is present (the rewrite fires and produces the expected IR), but the workload is too small to materialize a wall-clock difference. A workload that combines a broadcast with a wide downstream chain forcing the broadcasted tile to materialize across several mover boundaries is the natural next test.

F Detailed NCU Per-Tile Data

This appendix collects the per-tile NCU drill-downs for the main experiments. The narrative in Section 6 quotes the headline numbers from these tables (regs/thread, occupancy, spill load/store counts, DRAM throughput) at the WIN and REGR tiles only.

Table 7: NCU compact summary, $M = N = 8192$, `cat_trunc_2d`.

Tile	Pass	Dur (μs)	Reg	Occ %	DRAM %	Spill LD	Spill ST
(64, 64)	OFF	1515.4	80	73.1	80.9	0	0
(64, 64)	ON	1516.5	66	86.0	81.1	0	0
(128, 128)	OFF	1886.3	255	24.1	77.1	933,888	933,888
(128, 128)	ON	1477.3	218	24.8	83.7	0	0
(128, 256)	OFF	9381.3	64	94.1	73.7	14,508,032	14,778,368
(128, 256)	ON	2436.6	255	24.3	78.0	1,966,080	1,966,080
(256, 256)	OFF	9925.9	64	88.6	70.4	14,606,336	14,950,400
(256, 256)	ON	8710.1	64	89.9	68.8	12,251,136	12,423,168

Table 8: Warp-stall reasons (top, raw counts), `cat_trunc_2d`, $M = N = 8192$.

	(64, 64)		(128, 128)		(128, 256)		(256, 256)	
Reason	OFF	ON	OFF	ON	OFF	ON	OFF	ON
`lg_throttle`	10,182	13,173	4,417	3,037	139,652	10,334	148,353	123,478
`long_scoreboard`	4,805	8,416	11,436	24,245	39,715	37,858	41,472	59,885
`barrier`	10,065	7,116	2,716	2,450	23,408	2,808	22,948	6,421

Table 9: NCU compact summary, $M = N = 8192$, `deep_chain_2d`.

Tile	Pass	Dur (μs)	Reg	Occ %	DRAM %	Spill LD	Spill ST
(64, 64)	OFF	1230.9	72	85.4	83.7	0	0
(64, 64)	ON	1230.2	64	97.8	83.5	0	0
(128, 256)	OFF	9009.4	64	94.6	70.7	13,443,072	14,123,008
(128, 256)	ON	1232.7	203	24.8	83.4	0	0
(256, 256)	OFF	9117.9	64	89.1	73.1	14,123,008	14,770,176
(256, 256)	ON	7815.6	64	91.5	66.9	10,760,192	11,018,240

Table 10: NCU compact summary, $M = N = 8192$, `dual_quant_2d`.

Tile	Pass	Dur (μs)	Reg	Occ %	DRAM %	Spill LD	Spill ST
(64, 64)	OFF	2595.1	72	84.7	54.1	0	0
(64, 64)	ON	2601.4	68	86.3	55.4	0	0
(128, 128)	OFF	2664.3	255	24.0	64.9	1,507,328	1,294,336
(128, 128)	ON	2109.0	255	24.8	68.3	0	0
(128, 256)	OFF	9993.0	64	95.0	76.2	17,727,488	14,442,496
(128, 256)	ON	14700.8	64	94.9	63.8	20,766,720	20,078,592
(256, 256)	OFF	12731.3	64	90.1	70.0	19,742,720	18,841,600
(256, 256)	ON	12746.0	64	89.1	68.8	20,389,888	17,526,784

Table 11: NCU compact summary, $M = N = 8192$, multi-consumer `trans` (Pattern 5).

Tile	Pass	Dur (μs)	Reg	Occ %	Spill LD	Spill ST
(64, 64)	OFF	894.7	72	85.2	0	0
(64, 64)	ON	894.3	72	85.9	0	0
(128, 128)	OFF	952.5	168	36.6	0	0
(128, 128)	ON	951.4	168	37.1	0	0
(256, 128)	OFF	1104.5	255	24.8	483,328	417,792
(256, 128)	ON	1552.7	255	24.4	1,138,688	1,105,920

Table 12: Cross-kernel SASS opcode deltas at WIN tiles. Counts per thread; "→" marks with our optimization values. The bottom four rows (LDG/STG/F2F/F2I) are byte-identical between without our optimization and with our optimization in every column.

Opcode	`cat_trunc` (128, 128)	`deep_chain` (128, 256)	`mcc_3k` (64, 256)	`Pattern 7` (128, 256)
LDL family	→ 0	1543 → 0	197 → 52	→ 0
STL family	→ 0	1239 → 0	191 → 50	→ 0
BAR.SYNC	127 → 63	255 → 63	127 → 127	→ same
LDSM.16.M88.4	64 → 32	128 → 32	→ same	→ same
STS.128	64 → 32	128 → 32	→ same	→ same
PRMT	128 → 256	392 → 770	557 → 1219	→ 2×
LDG.E.128.SYS	64	128	64	128
STG.E.128.SYS	32	32	64	48
F2F.F16.F32	256	–	256	–
F2I.S16.TRUNC	–	512	256	512

Deep-chain SASS opcode diff at $(128, 256)$**:** Algorithmic global I/O preserved: `LDG.E.128.SYS` 128 = 128, `STG.E.128.SYS` 32 = 32, `F2I.S16.TRUNC.NTZ` 512 = 512. Eliminated: `LDL.LU + LDL.LU.64 + LDL.64 + LDL` (1543 → 0), `STL + STL.64` (1239 → 0). Reduced 4×: `BAR.SYNC` 255 → 63, `LDSM.16.M88.4` 128 → 32, `STS.128` 128 → 32. Doubled: `PRMT` 392 → 770 (register/permute staging replaces shared-memory staging).